2D/3D Visualization of Large-Scale Wind Field based on WebGL

Aijie Yao, Li Wang, Jun Li[a, *], Xiaoke Xia, Xing Jin, and Ning Jing
College of Electronic Science and Engineering, National University of Defense Technology, Changsha, China
[a]Corresponding author, e-mail: junli@nudt.edu.cn

ABSTRACT

With the increasing amount of data in the marine environment and the user's affluent requirements for visualization, traditional CPU-based visualization is becoming more and more difficult to complete specific tasks. To visualize the large-scale marine vector data, the particle system method is one of the most popular methods, but the data scale and complex calculations heavily restrict the efficiency of these CPU-based algorithms. This paper proposes a frame buffer visualization method with two strategies based on WebGL (FBO-W). To reduce the dense particle resulted by the increased visualization resolution, we dynamically set the number of particles based on different resolutions (PN-R); then, to alleviate the problem of unbalanced particle distribution during visualization, the strategy of dynamically adjusting particle drop rate based on particle velocity (DR-V) is incorporated. Experimental results show that the efficiency of data loading and rendering is greatly improved with higher visualization quality.

CCS CONCEPTS

• Social and professional topics~Professional topics~Management of computing and information systems~Project and people management~Project management techniques

KEYWORDS

Component; wind field data; NetCDF; particle system method; FBO-W; PN-R; DR-V.

ACM Reference format:

Aijie Yao, Li Wang, Jun Li, Xiaoke Xia, Xing Jin, and Ning Jing. 2020. 2D/3D Visualization of Large-Scale Wind Field based on WebGL. 2020 International Conference on Aviation Safety and Information Technology (ICASIT 2020), October 14--16, 2020, Weihai City, China. ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3434581.3434662

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

ICASIT 2020, October 14–16, 2020, Weihai City, China
© 2020 Association for Computing Machinery.
ACM ISBN 978-1-4503-7576-4/20/10…$15.00
https://doi.org/10.1145/3434581.3434662

1 Introduction

The visualization of large-scale marine environmental data is an important research topic in the field of computer graphics and virtual reality. Marine environmental data has the feature of large scale and difficult to analyze. Applying data visualization technology to marine environmental data is an important part of modern marine data analysis. Visualization of marine vector data has been extensively studied at home and abroad. Taking wind field data visualization as an example, the currently popular one is earth system (url: earth.nullschool.net), a well-known project developed by Cameron Beccario, which has comprehensive functions and excellent visualization effects. But the system has performance limitations: The number of wind particles that can be visualized is low, about 5k particles; because its calculation is based on the CPU, the cost of processing data per frame is very expensive, which makes the update of the data with a large delay (about 2s). Many teams have made many derivatives and improvements based on this system, such as Wind-JS from Esri, and two good weather service systems: Windy (url: windytv.com) and Ventusky (url: ventusky.com), but these are based on the CPU and have not achieved the integration of 2D and 3D. The better one done in China is Baidu's echarts-x, which is implemented based on WebGL, but it consumes performance seriously.

Efficient visualization of marine vector data requires a suitable visualization method and strong hardware support. Li Bo et.al proposed a spherical ray projection method to improve the traditional ray projection algorithm and effectively use the GPU [1]. He Yawen et.al proposed a process-oriented spatiotemporal visualization method for a large number of complex marine environment data, providing an integrated visualization solution for the data [2]. Peng Baoyun and others proposed data prefetching methods for large-scale flow field data. Based on parallel visualization and out-of-core technology, it greatly improved the visualization efficiency [3]. The above methods either require high computer hardware or fail to achieve the desired visualization.

In this paper, a frame buffer visualization method with two strategies based on WebGL (FBO-W) is proposed to improve the visualization quality and the efficiency of data loading and parsing. Based on the NetCDF format and particle system method, a strategy (PN-R) to dynamically set the number of particles is proposed, in which the number of particles is adapted according to the distance from the camera position. By this strategy, we improved the problem of the dense particle resulted by the increased visualization resolution. Another strategy (DR-V) to dynamically adjust the

particle drop rate is proposed, in which the drop rate is adapted according to different resolutions. By this strategy, we improved the problem of unbalanced distribution of random particles. Finally, we have proven the superiority of our method by experiment.

Our main contributions are threefold:

- Based on the particle system method, we propose the PN-R strategy, which improves the problem of too dense particles in the visualization;
- We propose the DR-V strategy, which improves the problem of unbalanced distribution of random particles in the visualization;
- We propose the FBO-W method with two strategies, which reduces data update delay and frequent data transmission between memory and GPU, and improves the system rendering efficiency.

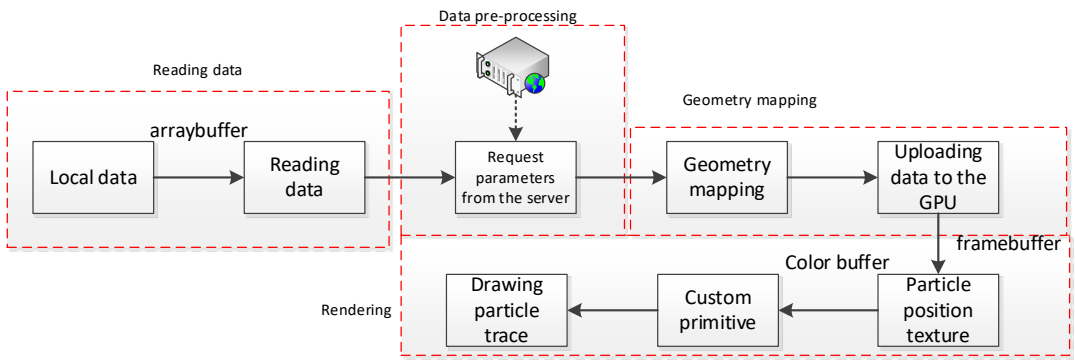


Fig. 1. Visualization pipeline.

2 Visualization Pipeline and Related Work

2.1 Visualization Pipeline

The basic flow visualization pipeline includes data reading, data pre-processing, geometry mapping, and rendering [7]. The detailed pipeline of visualization is descripted below and shown in Fig. 1.

Reading data: The website of wind field data has Global Forecast System (GFS), National Centers for Environmental Prediction (NECP), European Centre for Medium-Range Weather Forecasts (ECMWF) etc. The data source used in this system is GFS.

Data pre-processing: At this stage, the system uses Nodejs to calculate the maximum and minimum values required for wind field data visualization in parallel on the server, and stores it on the server, which is requested by the front end.

Geometry mapping: At this stage, this paper uses the particle system method. And the Cesium platform supports the use of basic graphics primitives, such as points, line and triangles.

Rendering: This article is based on the Cesium rendering engine to achieve efficient rendering on the GPU in the frame buffer. In addition, each rendering process requires interpolation of the wind field data.

2.2 Data Pre-processing

- When the browser reads data from the local or database, the traditional method is JSON format, but for large-scale data, JSON format [4] is not efficient enough. How to ensure efficient data transmission and parsing efficiency and achieve large-scale data rendering is the key to achieving fast visualization. For the problem, we use NetCDF format to store data and transmit and parse data in an ArrayBuffer format, which greatly reduces the time required for data transmission. This article compares the loading efficiency of JSON and NetCDF format.
- When visualizing the wind field data, you need to calculate the maximum and minimum speeds of the wind field data in the horizontal and vertical directions. Sometime, the parameter is not provided in the data, so it needs to be calculated. However, the collected global wind field data has a total of 259,200 particles. The browser needs lots of memory to calculate the maximum value, which may cause crash due to memory overflow. For this problem, we use the web worker technology natively supported by Nodejs to perform parallel computing on the server. Web worker of Nodejs is JavaScript that runs in the background and is independent of other scripts, which does not affect the performance of the browser.

2.3 Geometry Mapping

About particle system method [6-7], the streamline formula is as follows:

$$x_{stream(t;x_0,t_0)} = x_0 + \int_{t_0}^{t} v(x_{stream}(s;x_0,t_0),\tau)ds \quad (1)$$

$x(t;x_0,t_0)$ represents the vector line passing the point x_0 at time t_0 and $v(x,t)$ represents the speed (vector) at time t, and the speed depends only on the position and has nothing to do with time. t is a constant.

2.4 Rendering

- The difficulty of using the particle system method to visualize wind field data is that it requires lots of integral operations. Because the particle motion is dynamic, global particles must be calculated every frame. so how to calculate efficiently is the key to improve rendering efficiency. Compared with the advantages of CPU and GPU, the traditional CPU-based calculation method cannot meet the integral calculation of millions of particles. This paper uses a CPU + GPU collaborative computing model to perform 2D and 3D

visualization of wind field data.

• The rasterization of the fragment shader is the interpolation of pixels. The process of interpolating pixel into fragments will form a particle color gradient eventually, but the effect of WebGL's original interpolation is not satisfied enough. After the wind speed texture compression, it will cause blocky and pixel discontinuities. The rasterization of the fragment shader is shown in Fig. 2.

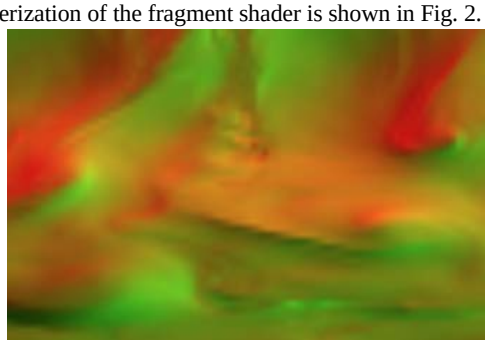


Fig. 2. Rasterization of the fragment shader

• To solve this problem, based on WebGL's original interpolation, we performed manual interpolation again. For 2D visualization, this paper uses a bilinear interpolation algorithm [11]; for 3D visualization, we use primitive TRIANGLE to instead POINT. To make the particle trajectory smoother, we choose bicubic interpolation algorithm [12]. Compared with bilinear interpolation, the latter's accuracy is higher, but its disadvantage huge interpolation calculation. Therefore, all interpolation calculations are performed parallelly on the GPU.

3 Proposed Methods

This section mainly introduces some problems and corresponding solutions in the visualization pipeline: To improve the data loading and rendering efficiency, we propose FBO-W method; To solve the unbalanced distribution of random particles, we propose DR-V strategy; To solve the problems of dense particles resulted by the increased visualization resolution, we propose PN-R strategy.

3.1 FBO-W Method

1) Problem: After the browser has parsed the wind field data, the particle wind speed is stored in the JavaScript array. At this time, the normal method is to randomly generate particle position information globally, and then transmit the particle wind speed information and random particle position information to the GPU [5]. The particle position of the next frame is calculated in the fragment shader, and the particle's color gradient is set according to the wind speed. Finally, the particle position was passed to the color buffer and drawed on the screen. Then we gets the pixel content from the canvas and uploads it to the GPU to draw the position of the particles in the next frame. However, this method requires that the pixel information on the canvas be obtained every frame and then passed to the GPU to calculate the position of the next frame. Compared to the internal transmission of the GPU, frequent transfer of data between the memory and the GPU greatly reduces the system performance and spends more time. Therefore, it should be used with caution, especially the reverse transfer from the CPU to the GPU.

2) Solution: To reduce the time of data transfers between memory and GPU, we use FBO-W strategy. Through off-screen rendering, the data only needs to be transmitted to the GPU once, which improves system performance. The specific steps are as follows: First, the wind speed of the particles is mapped into a texture by RGBA coding, and the randomly generated particle positions are also RGBA coded into a texture, and then transmitted to the fragment shader together. The wind speed and position of the particles are obtained through RGBA decoding, then the new position is calculated in the shader according to the wind speed, and then a new texture is encoded through RGBA. All of this is finished in the frame buffer.

About how to update the particle position information in the next frame, our solution is to use this new texture as the current state, and then to draw the new state into another texture in the next frame. We exchange these two textures every frame, then the texture of the latest particle position is obtained and passed to the color buffer.

About how to draw particle trajectories. For the rendering of 2D and 3D particle trajectories, this article chooses different primitives: 3D is TRIANGLES type, and 2D is POINT. The particle trajectory of 2D is drawn in a similar way to updating the particle position. In the current frame, the latest particle position texture in the color buffer is drawn to the screen. Then we use this texture as the background in the next frame and change its transparency to be slightly darker. Finally, we exchange the input/target texture in each frame, so that the latest particle position texture is always rendered to the screen, which makes the particle trajectory have a gradual pipeline of life cycle.

3.2 DR-V Strategy

1) Problem: The particle drop rate determines whether a randomly generated particle can be drawn on the screen. In other words, a particle which speed (normalized speed) is less than the particle drop rate is allowed to be displayed on the screen, otherwise abandoned. If the particle drop rate is set to a fixed value, randomly generated particles drawn on the screen are particles with lower speed. The randomly generated particles with higher speed are abandoned. Therefore, the original particles continue to be drawn along the current frame trajectory, which makes particles with faster speed much denser than areas without wind, because most of the areas with fast speed particles are not randomly generated.

2) Solution: For this problem, we propose DR-V strategy to balance the particle trajectory density. The strategy sets a weight dropRateBump on the speed of the particles to dynamically adjust the drop rate of the particles, and solves this problem by increasing the drop rate of the faster particles. Therefore, even if the random generation is fast, it can be drawn on the screen instead of discarded. The strategy formula is as follows:

$$particleDrop\mathrm{Rate} = dropRate + dropRateBump*nextSpeed \tag{2}$$

Among them, particleDropRate is the particle drop rate, that is, if the randomly generated wind speed is less than this value, the random particles are allowed to be drawn on the screen; dropRate is a fixed particle drop rate, which is the reference or minimum value of the drop rate; dropRateBump is the speed weight, which balances the denseness of the trajectory by increasing the drop rate of faster particles.

3.3 PN-R Strategy

1) Problem: In 3D wind field visualization, the resolution is changed by changing the height of the camera. As the resolution increases, the particles in the current visible range become denser and denser, which makes the visualization quality poor. The reason is that in this article, we judge whether the particles cross the boundary. That is, particles are generated only in the current visible range, and the number of particles is fixed.

2) Solution: For this problem, this paper proposes PN-R strategy. In addition, you can manually modify the maximum number of particles in the UI interface to set your satisfactory visualization effect. The strategy formula is as follows:

$$MaxParticles = \left(\frac{initMaxParticles}{initViewHeight}\right) * currentViewHeight + addedParticles \quad (3)$$

$$addedParticles = \begin{cases} 2000 & currentViewHeight > 20000 \\ 5000 & 15000 < currentViewHeight \leqslant 20000 \\ 7000 & 10000 < currentViewHeight \leqslant 15000 \\ 9000 & 5000 < currentViewHeight \leqslant 10000 \\ 6000 & 1000 < currentViewHeight \leqslant 5000 \\ 3000 & 100 < currentViewHeight \leqslant 1000 \\ 1000 & 10 < currentViewHeight \leqslant 100 \\ 200 & currentViewHeight \leqslant 10 \end{cases} \quad (4)$$

Among them, initMaxParticles is the number of particles initially, initViewHeight is the initial viewing height, currentViewHeight is the current viewing height, which varies with the camera height, and addedParticles is a segmentation constant.

4 Experimental results and analysis

To demonstrate the advantages of our system: significantly reducing data loading time and improving rendering efficiency, our experiments compare the efficiency between our method and representative CPU-based method, i.e. the online earth system (url: earth.nullschool.net) on the data set with JSON and NetCDF formats.

The data provided by the GFS is NetCDF format, and it has a size of 720 * 360 with 259200 grid points. The online earth system has a total of 4557 particles, and the format is JSON format.

Table 1. Experimental environment

Item	Description
CPU	4core*2, Intel(R) Core (TM) i5-7300HQ@2.50GHz
GPU	Intel(R) HD Graphics 630 NVIDIA GeForce GTX 1050
Memory	8GB
Software platform	Cesium 1.55
OS	Windows 10

4.1 Data Loading Efficiency Comparison Experiment

In order to test the effect of data format on data loading efficiency, and to improve the persuasiveness of the experiment, the wind field data at 7 atmospheres were selected. Each time the atmospheric pressure is changed, the time it takes to request and load data is shown in Fig. 3.

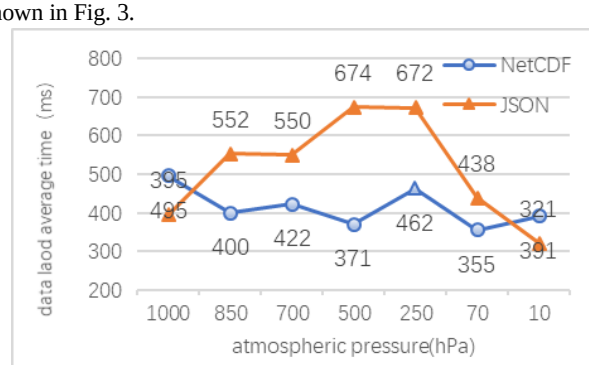


Fig. 3. Data loading efficiency comparison

As shown in Fig. 3 that under the condition of similar data loading time, the data size of the NetCDF format is about 56 times that of the JSON format, so the NetCDF format greatly improves the data loading efficiency.

4.2 Data Update Efficiency Comparison Experiment

To test the rendering efficiency of FBO-W method, our experiments compared the time spent in updating the data of our system and the earth system, including changing the number of particles, changing particle system parameters, and scaling the scene. The comparison of data update efficiency is depicted in Fig. 4.

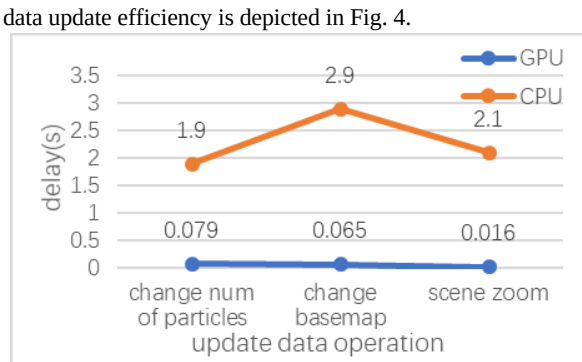


Fig. 4. Data update efficiency comparison

As shown in Fig. 4, with our system visualization method to render particles, the average delay of each data update is about 60ms; while with the earth system to render particles, each data update is delayed by about 2s, and the system performance is improved by about 33 times. The reason that the latter takes more time to update the data each time is that each update requires interpolation for all particles, and the interpolation calculation is performed on the CPU. The reason that our system visualizes a large number of particles and takes less time to update is that all the interpolation and integration calculations of this system are performed on the GPU.

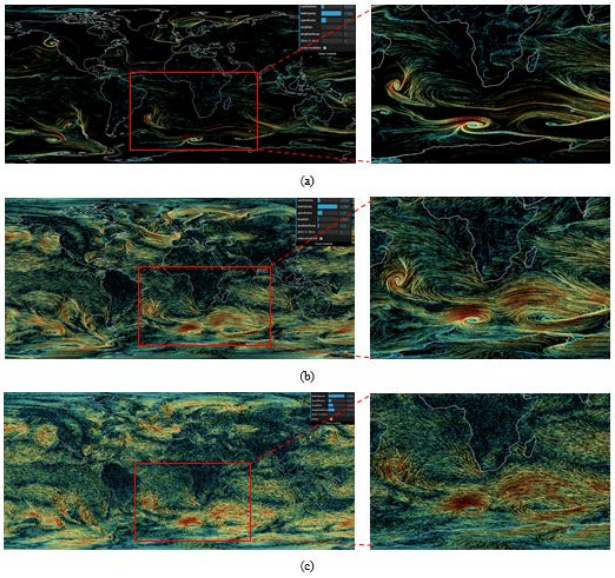
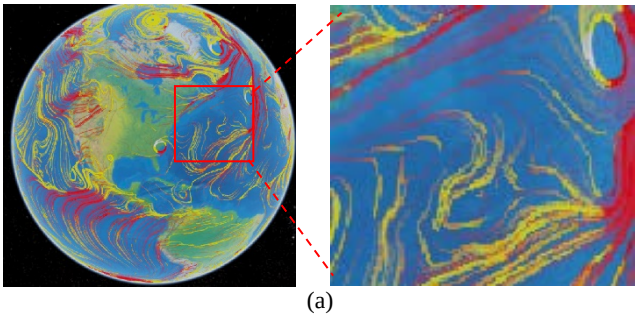


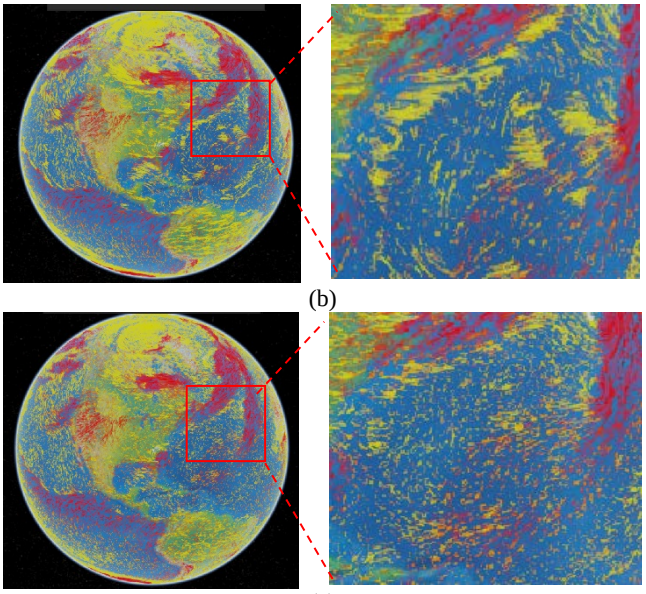
Fig. 5. Change particle drop rate with our strategy in 2D (a). drop rate is zero and local zoom. (b). drop rate is low and local zoom. (c). drop rate is high and local zoom

4.3 Improvements of Visualization Effect

1) DR-V strategy: After dynamically adjusting the particle drop rate, the visualization of 2D is shown in Fig. 5 and 3D in Fig. 6. We can find that DR-V strategy can well balance the fast and slow track density of particles.



(a)

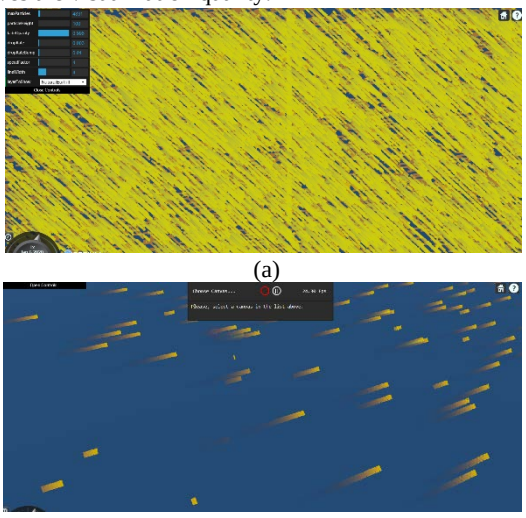


(b)

(c)

Fig. 6. Change particle drop rate with our strategy in 3D (a). drop rate is zero and local zoom. (b). drop rate is low and local zoom. (c). drop rate is high and local zoom

2) PN-R strategy: The contrast effect with or without our strategy is shown in Fig. 7. We can find that PN-R strategy can well set proper number of particle based on different resolution, which improves the visualization quality.



(a)

(b)

Fig. 7. Contrast effect in 3D with or without our strategy. (a). the effect without our strategy. (b). the effect with our strategy

5 Cloclusion and Future Work

For the 2D and 3D visualization of marine vector data, long data loading time, low data update efficiency, and poor visualization effects are heavily limiting the application and analysis of marine

vector data.

This paper proposes a frame buffer visualization method with two strategies based on WebGL (FBO-W), which improves the visualization quality and the efficiency of data loading and rendering. We dynamically set the number of particles based on different resolutions (PN-R), which reduces the dense particle resulted by the increased visualization resolution; then, we dynamically adjust particle drop rate based on particle velocity (DR-V), which alleviates the problem of unbalanced particle distribution during visualization. Experimental results demonstrate the superiority of our method by effectively rendering the wind field data with one million particles at 60fps.

In the future, our work will be focused on how to dynamically set particle number based on our focus. In other words, we just need to make the places that we care about appear clear and that we don't care about appear blurred. This new idea will further reduce the number of particles to be rendered and improve the visualization quality.

ACKNOWLEDGMENTS

This research was supported by the Natural Science Foundation of China under Project 61806211. the data was supported by the GFS.

REFERENCES

[1] Li, Bo, et al. "GPU accelerated marine data visualization method." Journal of Ocean University of China 13.6 (2014): 964-970.

[2] Yawen, He, et al. "Web-based visualization of marine environment data." 2010 18th International Conference on Geoinformatics. IEEE, 2010.

[3] Peng, B., W. Wang, and S. Li. "Data Prefetching in Streamline Visualization of Large Scale Flow Field." Journal of Computer-Aided Design and Computer Graphics 28.3 (2016): 464-470.

[4] Wang, Xiang-Hong, et al. "Visualization of marine environment data based on NetCDF data model." Science of Surveying and Mapping 38.2 (2013): 59-61.

[5] Xin Wenpeng, Fang Jing, Xia Wei. "The design and implementation of ocean visualization system based on WebGL. " Marine Information, 2018, 33(3): 44-48.

[6] WANG Huaihui, LI Sikun. "Research on Key Features of Feature-Based Texture Visualization of Complex Flow Field."(2015).

[7] Li, S. K., et al. "Large-Scale Flow Field Scientific Visualization." Beijing: National Defense Industry Press (2013).

[8] LIU Zhen-dong;LIU Hai-xing;SU Tian-yun;JIA Zhen;LI Xin-fang, "Dynamic simulation and visualization of ocean current based on LIC". Marine Science Bulletin, 2019, 38(2): 167-172

[9] Berger, Siegrun, and Eduard Gröller. "Color-table animation of fast oriented line integral convolution for vector field visualization." (2000).

[10] Liu, Zhanping, and Robert J. Moorhead. "Accelerated unsteady flow line integral convolution." IEEE Transactions on Visualization and Computer Graphics 11.2 (2005): 113-125.

[11] Li, Wei, Yuanfu Xie, and Guijun Han. "A theoretical study of the multigrid three-dimensional variational data assimilation scheme using a simple bilinear interpolation algorithm." Acta Oceanologica Sinica 32.3 (2013): 80-87.

[12] Aràndiga, Francesc. "A nonlinear algorithm for monotone piecewise bicubic interpolation." Applied Mathematics and Computation 272 (2016): 100-113.